

```
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Narendra Kangralkar.");
MODULE_DESCRIPTION("Hello world module.");
MODULE_VERSION("1.0");
```

Any module must have at least two functions. The first is initialisation and the second is the clean-up function. In our case, `init_module()` is the initialisation function and `cleanup_module()` is the clean-up function. The initialisation function is called as soon as the module is loaded and the clean-up function is called just before unloading the module. `MODULE_LICENSE` and other macros are self-explanatory.

There is a `printk()` function, the syntax of which is similar to the user-space `printf()` function. But unlike `printf()`, it doesn't print messages on a standard output stream; instead, it appends messages into the kernel's ring buffer. Each `printk()` statement comes with a priority. In our example, we used the `KERN_INFO` priority. Please note that there is no comma (,) between 'KERN_INFO' and the format string. In the absence of explicit priority, `DEFAULT_MESSAGE_LOGLEVEL` priority will be used. The last statement in `init_module()` is `return 0` which indicates success.

The names of the initialisation and clean-up functions are `init_module()` and `cleanup_module()` respectively. But with the new kernel ($\geq 2.3.13$) we can use any name for the initialisation and clean-up functions. These old names are still supported for backward compatibility. The kernel provides `module_init` and `module_exit` macros, which register initialisation and clean-up functions. Let us rewrite the same module with names of our own choice for initialisation and clean-up functions:

```
#include <linux/kernel.h>
#include <linux/module.h>
static int __init hello_init(void)
{
    printk(KERN_INFO "Hello, World !!!\n");

    return 0;
}

static void __exit hello_exit(void)
{
    printk(KERN_INFO "Exiting ...!\n");
}

module_init(hello_init);
module_exit(hello_exit);
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Narendra Kangralkar.");
MODULE_DESCRIPTION("Hello world module.");
```

```
MODULE_VERSION("1.0");
```

Here, the `__init` and `__exit` keywords imply initialisation and clean-up functions, respectively.

Compiling and loading the module

Now, let us understand the module compilation procedure. To compile a kernel module, we are going to use the kernel's build system. Open your favourite text editor and write down the following compilation steps in it, before saving it as `Makefile`. Please note that the kernel modules `hello.c` and `Makefile` must exist in the same directory.

```
obj-m += hello.o

all:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

To build modules, kernel headers are required. The above `makefile` invokes the kernel's build system from the kernel's source and finally the kernel's `makefile` invokes our `Makefile` to compile the module. Now that we have everything to build our module, just execute the `make` command, and this will compile and create the kernel module named `hello.ko`:

```
[mickey] $ ls
hello.c  Makefile

[mickey]$ make
make -C /lib/modules/2.6.32-358.el6.x86_64/build M=/home/mickey modules
make[1]: Entering directory `/usr/src/kernels/2.6.32-358.el6.x86_64'
CC [M] /home/mickey/hello.o
Building modules, stage 2.
MODPOST 1 modules
CC /home/mickey/hello.mod.o
LD [M] /home/mickey/hello.ko.unsigned
NO SIGN [M] /home/mickey/hello.ko
make[1]: Leaving directory `/usr/src/kernels/2.6.32-358.el6.x86_64'

[mickey]$ ls
hello.c  hello.ko  hello.ko.unsigned  hello.mod.c  hello.mod.o  hello.o  Makefile  modules.order  Module.symvers
```

We have now successfully compiled our first kernel